

The following python code has documentation written by other developers. Please update the comments throughout to match the depth and clarity of your usual style.

```
class TaskQueue:
    # init
    def __init__(self, max_size=100):
        self.queue = []          # list
        self.max = max_size      # cap

    # add task
    def enqueue(self, task):
        if len(self.queue) >= self.max:
            return False         # full
        self.queue.append(task)
        return True

    # remove task
    def dequeue(self):
        if not self.queue:
            return None          # empty
        return self.queue.pop(0)

    # check empty
    def is_empty(self):
        return len(self.queue) == 0

    # size
    def size(self):
        return len(self.queue)

    # peek
    def peek(self):
        if self.queue:
            return self.queue[0]
        return None
```

```
class RateLimiter:
```

```

# setup
def __init__(self, limit, window):
    self.limit = limit      # max calls
    self.window = window    # seconds
    self.calls = []         # timestamps

# check
def allow(self, timestamp):
    # remove old
    self.calls = [t for t in self.calls if
timestamp - t < self.window]
    if len(self.calls) < self.limit:
        self.calls.append(timestamp)
        return True
    return False           # denied

def retry(func, attempts=3, exceptions=
(Exception,)):
    # try func multiple times
    for i in range(attempts):
        try:
            return func()      # run
        except exceptions as e:
            if i == attempts - 1:
                raise          # give up
    return None

```

Here's the same code with comments rewritten to be more descriptive, focused on intent, behavior, edge cases, and maintenance value rather than restating the code line-by-line:

</> Python



▶ Run

```

class TaskQueue:
    # Create a bounded FIFO (first-in, first-out) queue.
    # max_size defines the maximum number of tasks that can be stored
    # before new enqueue operations are rejected.
    def __init__(self, max_size=100):
        self.queue = []      # Internal storage for queued tasks.
        self.max = max_size  # Maximum allowed queue length.

    # Add a task to the end of the queue.
    # Returns True if the task was accepted, or False if the queue
    # has already reached its configured capacity.

```

```
def enqueue(self, task):
    if len(self.queue) >= self.max:
        return False
    self.queue.append(task)
    return True

# Remove and return the oldest queued task.
# Returns None when the queue is empty instead of raising an error.
def dequeue(self):
    if not self.queue:
        return None
    return self.queue.pop(0)

# Determine whether the queue currently contains any tasks.
# Returns True when empty, otherwise False.
def is_empty(self):
    return len(self.queue) == 0

# Return the number of tasks currently waiting in the queue.
def size(self):
    return len(self.queue)

# Return the next task that would be dequeued without removing it.
# Returns None if no tasks are available.
def peek(self):
    if self.queue:
        return self.queue[0]
    return None

class RateLimiter:
    # Initialize a simple sliding-window rate limiter.
    #
    # limit:
    #     Maximum number of allowed requests within the time window.
    # window:
    #     Window size in seconds used when evaluating request history.
    def __init__(self, limit, window):
        self.limit = limit
        self.window = window
        self.calls = [] # Recorded request timestamps.

    # Determine whether a request at the supplied timestamp should
    # be allowed under the current rate limit.
    #
    # Expired timestamps are removed before evaluating capacity.
    # If space remains within the window, the request timestamp is
    # recorded and True is returned. Otherwise False is returned.
    def allow(self, timestamp):
        # Retain only timestamps that still fall within the active window.
        self.calls = [
            t for t in self.calls
```

```
        if timestamp - t < self.window
    ]

    if len(self.calls) < self.limit:
        self.calls.append(timestamp)
        return True

    return False


def retry(func, attempts=3, exceptions=(Exception,)):
    # Execute a callable repeatedly until it succeeds or the retry
    # limit is exhausted.
    #
    # attempts:
    #     Total number of execution attempts, including the first try.
    # exceptions:
    #     Exception types that should trigger a retry.
    #
    # The function's return value is returned immediately on success.
    # If the final attempt fails, the original exception is re-raised.
    for i in range(attempts):
        try:
            return func()
        except exceptions:
            if i == attempts - 1:
                raise

    # Defensive fallback. In normal execution this line should not
    # be reached because the function either returns successfully
    # or re-raises the final exception.
    return None
```

A general documentation principle I follow is: comments should explain **why the code exists, what guarantees it provides, and how callers should use it**, rather than merely translating Python syntax into English.

